

```

import cv2
from google.colab import files
import os

# IMPORTANT: You need to upload your image to Colab and update this path.
# Here's how you can do it:
# 1. Click on the folder icon on the left sidebar in Colab.
# 2. Click on the 'Upload to session storage' icon (looks like an arrow pointing up).
# 3. Select your image file (e.g., 'q1.jpg').
# 4. Once uploaded, use its name directly as the path.

# For example, if you upload 'q1.jpg':
image_path = "q1.jpg" # <--- Update this line after uploading your image!

# Alternatively, you can use files.upload() directly in a code cell:
# uploaded = files.upload()
# if uploaded:
#     image_path = list(uploaded.keys())[0]
#     print(f"Image '{image_path}' uploaded successfully.")
# else:
#     image_path = None
#     print("No image uploaded.")

img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from '{image_path}'.\nPlease ensure you have uploaded the image to Colab and the 'image_
else:
    small = cv2.resize(img, (100, 100), interpolation=cv2.INTER_NEAREST)

    sampled = cv2.resize(small, (img.shape[1], img.shape[0]),
                        interpolation=cv2.INTER_NEAREST)

    corrected = cv2.resize(small, (img.shape[1], img.shape[0]),
                        interpolation=cv2.INTER_CUBIC)

    # In Colab, cv2.imshow does not work directly as it requires a graphical display.
    # You'll need to convert the images to a format that can be displayed in the notebook.
    # For displaying images in Colab, use matplotlib or cv2_imshow from google.colab.patches.
    # Let's use matplotlib for demonstration.
    from matplotlib import pyplot as plt

    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(cv2.cvtColor(sampled, cv2.COLOR_BGR2RGB))
    plt.title("Improper Sampling")
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(cv2.cvtColor(corrected, cv2.COLOR_BGR2RGB))
    plt.title("Corrected Image")
    plt.axis('off')

    plt.show()

    # The cv2.waitKey(0) and cv2.destroyAllWindows() are not needed for matplotlib display in Colab.
    # If you were running this on a local machine with a graphical environment, they would pause the display.

```

Original Image



Improper Sampling



Corrected Image



```
import cv2
from matplotlib import pyplot as plt

# The image 'q2.jpg' has been uploaded to the Colab environment.
image_path = "q2.jpg" # Corrected path to the uploaded image

img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    enhanced = cv2.equalizeHist(gray)

    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for matplotlib
    plt.title("Original Low-Light Image")
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(gray, cmap='gray') # Grayscale images can be displayed directly
    plt.title("Gray Image")
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(enhanced, cmap='gray')
    plt.title("Enhanced Image")
    plt.axis('off')

    plt.show()

# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.
```

Original Low-Light Image



Gray Image



Enhanced Image



```
import cv2
from matplotlib import pyplot as plt # Import matplotlib for image display

# Correct the image path to 'q4.jpg' as it's now uploaded to Colab
```

```

image_path = "q4.jpg"
img = cv2.imread(image_path, 0)

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded to Colab.")
else:
    low_pixel = cv2.resize(img, (128, 128))
    low_pixel = cv2.resize(low_pixel,
                           (img.shape[1], img.shape[0]))

    quantized = (img // 32) * 32

    # Use matplotlib for displaying images in Colab instead of cv2.imshow
    plt.figure(figsize=(18, 6))

    plt.subplot(1, 3, 1)
    plt.imshow(img, cmap='gray') # Grayscale images can be displayed directly
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(low_pixel, cmap='gray')
    plt.title("Low Pixel Resolution")
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap='gray')
    plt.title("Reduced Intensity Resolution")
    plt.axis('off')

    plt.show()

# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.

```

Original Image



Low Pixel Resolution



```

import cv2
from matplotlib import pyplot as plt # Import matplotlib for image display

# Correct the image path to 'q5.jpg' as it's now uploaded to Colab
image_path = "q5.jpg"
img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded to Colab.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    edges = cv2.Canny(gray, 100, 200)

```

```
# Use matplotlib for displaying images in Colab instead of cv2.imshow
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for matplotlib
plt.title("Original Surveillance Image")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(gray, cmap='gray') # Grayscale images can be displayed directly
plt.title("Gray Image")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(edges, cmap='gray')
plt.title("Detected Edges")
plt.axis('off')

plt.show()

# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.
```

Original Surveillance Image



Gray Image



Detected Edges



```
import cv2
from matplotlib import pyplot as plt # Import matplotlib for image display

# Correct the image path to 'q3.jpg' as it's now uploaded to Colab
image_path = "q3.jpg"
img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded to Colab.")
else:
    small = cv2.resize(img, (120, 120))

    alias = cv2.resize(small,
                        (img.shape[1], img.shape[0]),
                        interpolation=cv2.INTER_NEAREST)

    filtered = cv2.GaussianBlur(alias, (5, 5), 0)

# Use matplotlib for displaying images in Colab instead of cv2.imshow
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for matplotlib
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 3, 2)
```

```
plt.imshow(cv2.cvtColor(alias, cv2.COLOR_BGR2RGB))
plt.title("Aliased Image")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(filtered, cv2.COLOR_BGR2RGB))
plt.title("Filtered Image")
plt.axis('off')

plt.show()

# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.
```

Original Image



Aliased Image



Filtered Image



```
import cv2
from google.colab import files
from google.colab.patches import cv2_imshow

# Upload image
uploaded = files.upload()

img = cv2.imread("q6.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Improve image quality
enhanced = cv2.equalizeHist(gray)

print("Original Image")
cv2_imshow(img)

print("Enhanced Image")
cv2_imshow(enhanced)
```


Choose Files q6.jpg

q6.jpg(image/jpeg) - 250166 bytes, last modified: 7/23/2026 - 100% done

Saving q6.jpg to q6 (1).jpg

Original Image



```
import cv2
from matplotlib import pyplot as plt # Import matplotlib for image display

# Correct the image path to 'q7.jpg' as it's now uploaded to Colab
image_path = "q7.jpg"
img = cv2.imread(image_path, 0) # Load as grayscale

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded to Colab.")
else:
    # 4-bit Quantization
    quantized = (img // 16) * 16

    # Use matplotlib for displaying images in Colab instead of cv2.imshow
    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.imshow(img, cmap='gray') # Display grayscale image
    plt.title("Original Grayscale Image")
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(quantized, cmap='gray')
    plt.title("Quantized Image")
    plt.axis('off')

    plt.show()

# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.
```

Original Grayscale Image



Quantized Image



```
import cv2
from matplotlib import pyplot as plt # Import matplotlib for image display

# Correct the image path to 'q8.jpg' as it's now uploaded to Colab
image_path = "q8.jpg"
img = cv2.imread(image_path)
```

```

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded to Colab.")
else:
    # Remove Noise (make sure image is color if using Colored denoise)
    # If the image was loaded in grayscale, you might need to convert it or use fastNlMeansDenoising
    denoise = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 21)

    # Use matplotlib for displaying images in Colab instead of cv2.imshow
    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for matplotlib
    plt.title("Original Noisy Image")
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(cv2.cvtColor(denoise, cv2.COLOR_BGR2RGB))
    plt.title("Denoised Image")
    plt.axis('off')

    plt.show()

    # cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.

```

Original Noisy Image



Denoised Image



```

import cv2
from matplotlib import pyplot as plt # Import matplotlib for image display

# Correct the image path to 'q9.jpg' as it's now uploaded to Colab
image_path = "q9.jpg"
img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded to Colab.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    face = cv2.CascadeClassifier(cv2.data.haarcascades +
                                "haarcascade_frontalface_default.xml")

    faces = face.detectMultiScale(gray, 1.1, 4)

    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # Use matplotlib for displaying images in Colab instead of cv2.imshow
    plt.figure(figsize=(8, 6))
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for matplotlib
    plt.title("Face Detection")
    plt.axis('off')
    plt.show()

    # cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.

```

Face Detection



```
import cv2
from matplotlib import pyplot as plt # Import matplotlib for image display

# Correct the image path to 'q10.jpg' as it's now uploaded to Colab
image_path = "q10.jpg"
img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from '{image_path}'. Please ensure the image is uploaded to Colab.")
else:
    # Reduce Resolution
    low = cv2.resize(img, (64, 64))

    # Restore Image
    restored = cv2.resize(low,
                          (img.shape[1], img.shape[0]),
                          interpolation=cv2.INTER_CUBIC)

    # Use matplotlib for displaying images in Colab instead of cv2.imshow
    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for matplotlib
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(cv2.cvtColor(restored, cv2.COLOR_BGR2RGB))
    plt.title("Low Resolution Image")
    plt.axis('off')

    plt.show()

# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed when using matplotlib in Colab.
```

Original Image



Low Resolution Image

